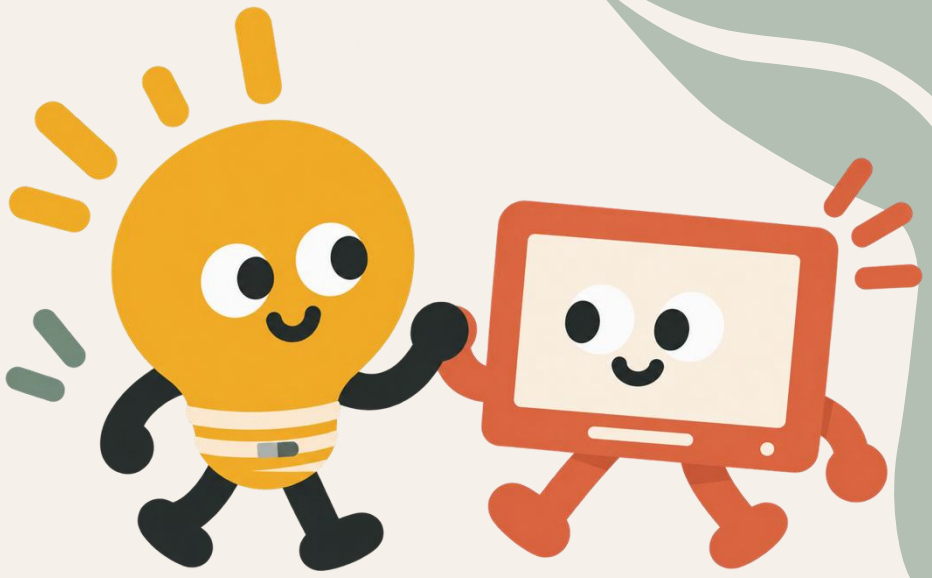
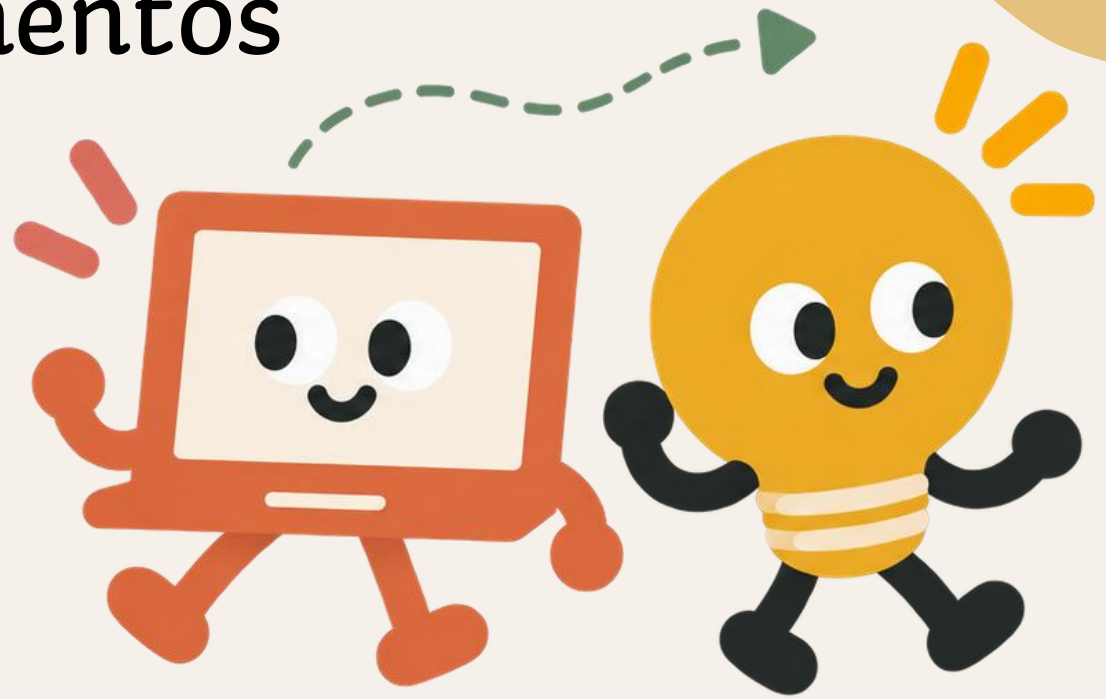
Módulo 5

Taller teórico-práctico
Del dataset a U-Net: segmentación de imágenes con Deep Learning

→ De la convolución a U-Net

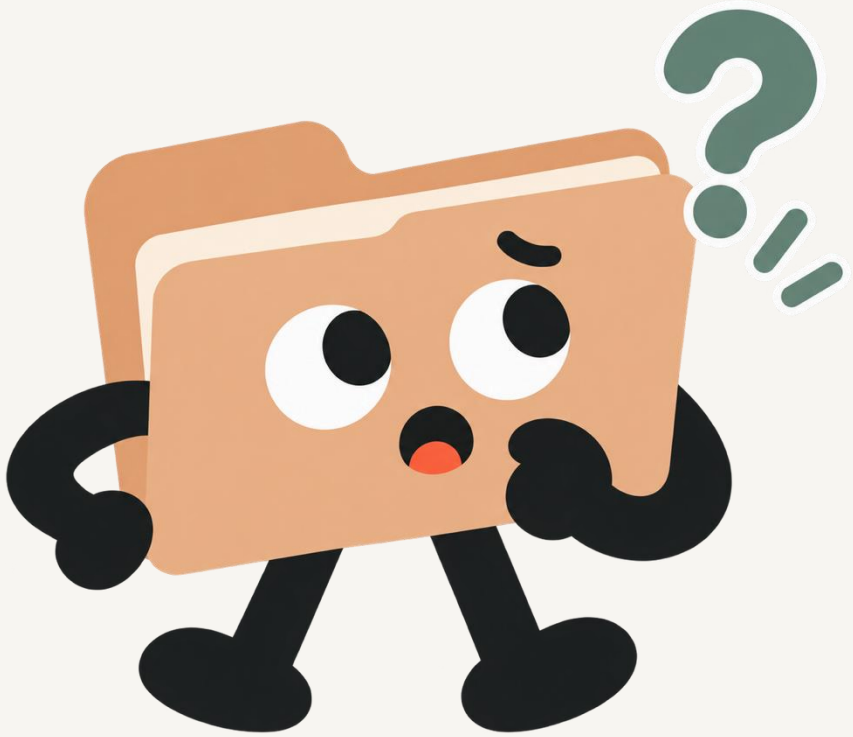


Convolución y fundamentos de las CNN



Tenemos un tensor de imágenes y otro de máscaras.

¿Qué operaciones debe realizar una red neuronal para transformar el primero en una predicción con la estructura espacial del segundo?



Conceptos clave de la convolución

Imagen de entrada (Input Image)

Conjunto de valores de píxeles que contiene la información visual que procesará la red neuronal. Puede tener uno o varios canales, como RGB.



Imagen RGB
(H × W × 3)

Canal R		
255	120	75
200	95	60
180	80	50

Canal G		
120	200	90
100	180	70
90	150	60

Canal B		
70	110	200
60	90	180
50	80	160

Cada canal es una matriz de valores de píxeles (H × W).

Kernel o filtro

Matriz de pesos que se desplaza sobre la imagen de entrada para detectar patrones locales, como bordes, texturas o formas.
Sus valores se aprenden durante el entrenamiento de una CNN.

1	0	0
0	0	−3
2	0	−1

Kernel (3 × 3)
Pesos aprendibles

Un kernel puede estar diseñado para detectar, por ejemplo, bordes verticales, horizontales, texturas o formas especificas.



Mapa de características (Feature Map)

Resultado de aplicar un kernel a la entrada mediante una operación de convolución. Representa la respuesta del filtro ante determinados patrones visuales.

−1	0	1
−2	−7	0
−6	−3	0

Mapa de características (3 × 3)

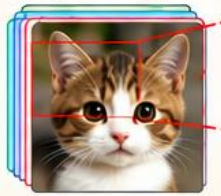
Cada valor indica qué tan bien coincide el patrón del kernel con la región de la imagen en esa posición.



Relación entre los tres conceptos

El kernel recorre la imagen y calcula una respuesta en cada posición, generando un mapa de características.
Cada filtro produce un mapa de características, y distintos filtros permiten identificar diferentes patrones visuales.

Imagen de entrada
(H × W × C)



1	2	3
0	1	2
2	1	0

Región de la imagen (3 × 3)

×

1	0	0
0	0	−3
2	0	−1

Kernel (3 × 3)

→

Convolución

Se realizan multiplicaciones elemento a elemento y se suman los resultados.

El kernel se desplaza por la imagen para generar las casillas del mapa de características.

→

Mapa de características
(H_{out} × W_{out})

−1	0	1
−2	−7	0
−6	−3	0

Varios filtros, distintos mapas de características

Cada filtro aprende a detectar un tipo de patrón.

−1	0	1
−1	0	1
−1	0	1

→ 

Filtro de borde vertical

1	1	1
0	0	0
−1	−1	−1

→ 

Filtro de borde horizontal

0	1	0
1	−4	1
0	1	0

→ 

Filtro de detección de contornos

En una CNN: características cada vez más complejas

Los mapas de características generados en una capa convolucional pueden convertirse en las entradas de las siguientes capas, permitiendo aprender patrones cada vez más complejos.



Desplazamiento del kernel sobre la imagen

El kernel se mueve sobre la imagen (stride = 1), calculando un valor en cada posición válida.
Con una imagen de 5 × 5 y un kernel de 3 × 3 (sin padding), se obtienen 3 × 3 posiciomes de salida.

Kernel (3 × 3)

1	0	0
0	0	-3
2	0	-1

Imagen de entrada (5 × 5)

	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$
$i=0$	1	2	3	0	1
$i=1$	0	1	2	1	0
$i=2$	2	1	0	1	2
$i=3$	1	0	1	2	1
$i=4$	0	1	2	3	1

Las 9 posiciones del kernel y las regiones de la imagen

1 Posición (0, 0)

1	2	3	0	1
0	1	2	1	0
2	1	0	1	2
1	0	1	2	1
0	1	2	3	1

2 Posición (0, 1)

1	2	3	0	1
0	1	2	1	0
2	1	0	1	2
1	0	1	2	1
0	1	2	3	1

3 Posición (0, 2)

1	2	3	0	1
0	1	2	1	0
2	1	0	1	2
1	0	1	2	1
0	1	2	3	1

4 Posición (1, 0)

1	2	3	0	1
0	1	2	1	0
2	1	0	1	2
1	0	1	2	1
0	1	2	3	1

5 Posición (1, 1)

1	2	3	0	1
0	1	2	1	0
2	1	0	1	2
1	0	1	2	1
0	1	2	3	1

6 Posición (1, 2)

1	2	3	0	1
0	1	2	1	0
2	1	0	1	2
1	0	1	2	1
0	1	2	3	1

7 Posición (2, 0)

1	2	3	0	1
0	1	2	1	0
2	1	0	1	2
1	0	1	2	1
0	1	2	3	1

8 Posición (2, 1)

1	2	3	0	1
0	1	2	1	0
2	1	0	1	2
1	0	1	2	1
0	1	2	3	1

9 Posición (2, 2)

1	2	3	0	1
0	1	2	1	0
2	1	0	1	2
1	0	1	2	1
0	1	2	3	1

Cálculo de la convolución en la posición (0, 0)

En la posición (0, 0), el kernel se aplica sobre la región superior izquierda de la imagen.
Se multiplican los valores correspondientes elemento a elemento y se suman los resultados.

1 Imagen de entrada (5 × 5)

i \ j	0	1	2	3	4
0	1	2	3	0	1
1	0	1	2	1	0
2	2	1	0	1	2
3	1	0	1	2	1
4	0	1	2	3	1

Imagen X de tamaño 5 × 5

Para la posición (0, 0), se utiliza la región de 3 × 3 en la esquina superior izquierda.

2 Kernel (3 × 3)

1	0	0
0	0	−3
2	0	−1

Kernel W de tamaño 3 × 3

3 Multiplicaciones elemento a elemento

Se multiplican los valores de la región de la imagen con los valores correspondientes del kernel.

Región de la imagen X(0:3, 0:3)

1	2	3
0	1	2
2	1	0

×

Kernel W(0:3, 0:3)

1	0	0
0	0	−3
2	0	−1

=

Productos elemento a elemento

1×1	2×0	3×0
0×0	1×0	2×(−3)
2×2	1×0	0×(−1)

4 Sustitución en la fórmula

La operación (sin bias) se define como:

$$Y(i,j) = \sum_{m=0}^{K_h-1} \sum_{n=0}^{K_w-1} W(m,n)X(i+m, j+n)$$

Para la posición (0, 0) con un kernel de 3 × 3 ($K_h = K_w = 3$):

$$Y(0,0) = \sum_{m=0}^{2} \sum_{n=0}^{2} W(m,n)X(0+m, 0+n)$$



Cada término de la suma corresponde a la multiplicación de un elemento de la imagen con el elemento del kernel en la misma posición.

5 Cálculo paso a paso

Sustituimos los valores de la región y del kernel:

$$Y(0,0) = \underbrace{(1)(1)}_{(0,0)} + \underbrace{(2)(0)}_{(0,1)} + \underbrace{(3)(0)}_{(0,2)} + \underbrace{(0)(0)}_{(1,0)} + \underbrace{(1)(0)}_{(1,1)} + \underbrace{(2)(-3)}_{(1,2)} + \underbrace{(2)(2)}_{(2,0)} + \underbrace{(1)(0)}_{(2,1)} + \underbrace{(0)(-1)}_{(2,2)}$$

6 Suma de los resultados

Realizamos las multiplicaciones:

$$Y(0,0) = \underbrace{1}_{(0,0)} + \underbrace{0}_{(0,1)} + \underbrace{0}_{(0,2)} = 1$$

$$+ \underbrace{0}_{(1,0)} + \underbrace{0}_{(1,1)} - \underbrace{6}_{(1,2)} = -6$$

$$+ \underbrace{4}_{(2,0)} + \underbrace{0}_{(2,1)} + \underbrace{0}_{(2,2)} = 4$$

Suma total:

$$Y(0,0) = 1 - 6 + 4 = -1$$

Parámetros de una capa convolucional

1 Weight (kernel)

Matriz de pesos que se desplaza sobre la imagen para detectar patrones locales, como bordes, texturas o formas. Sus valores se aprenden durante el entrenamiento de la red.

1	0	0
0	0	−3
2	0	−1

Kernel (3 × 3)
W(m, n)
(pesos aprendibles)

Cada valor del kernel indica la importancia de un píxel en la región de la imagen para el patrón que se quiere detectar.

2 Bias

Valor adicional que se suma al resultado de la convolución. Permite desplazar la respuesta del filtro y aumenta la capacidad de representación de la capa. Es un parámetro entrenable (usualmente un valor por filtro).

$$y(i,j) = \sum_{m=0}^{K_h-1} \sum_{n=0}^{K_w-1} W(m,n)X(i+m,j+n) + b$$

- $W(m,n)$ — pesos del kernel (weight)
- $X(i+m,j+n)$ — valores de la imagen en la región local
- b — bias (escalar)

3 Stride (paso de desplazamiento)

Define cuántas posiciones avanza el kernel sobre la imagen en cada movimiento. Un stride mayor produce mapas de características de menor resolución espacial.

Stride = 1

El kernel avanza una casilla por movimiento.

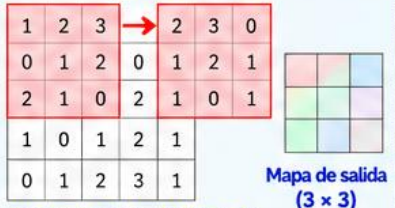


Imagen de entrada (5 × 5)
Kernel: 3 × 3
Stride: 1
9 posiciones evaluadas

Stride = 2

El kernel avanza dos casillas por movimiento.

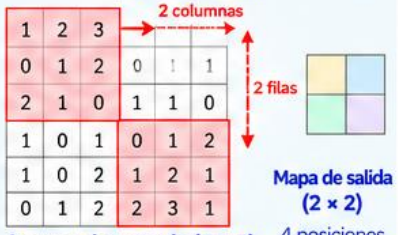


Imagen de entrada (5 × 5)
Kernel: 3 × 3
Stride: 2
4 posiciones evaluadas

4 Padding (relleno)

Controla el tratamiento de los bordes agregando un marco de ceros (u otros valores) alrededor de la imagen de entrada. Permite mantener o ajustar las dimensiones de salida.

Sin padding (p = 0)

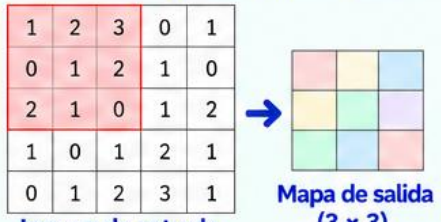


Imagen de entrada (5 × 5)

Con padding (p = 1)

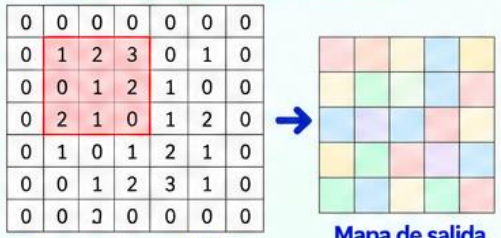


Imagen con padding (7 × 7)

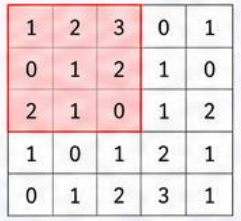
5 Dilation (dilatación)

Permite ampliar el campo receptivo del kernel sin aumentar su número de pesos. Los elementos del kernel se aplican sobre píxeles más separados, dejando espacios (dilataciones) entre ellos.

Dilation = 1 (convencional)

1	0	0
0	0	−3
2	0	−1

Kernel (3 × 3)
(dilation = 1)

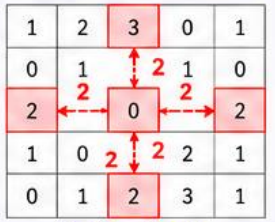


Región utilizada
(píxeles contiguos)

Dilation = 2

1	0	0
0	0	−3
2	0	−1

Kernel (3 × 3)
(dilation = 2)



Región utilizada
(píxeles separados con dilatación = 2)

Resumen

- **Weight (kernel):** determina qué patrones se buscan en la entrada.
- **Bias:** se suma al resultado de la convolución y es un parámetro entrenable.
- **Stride:** controla cuánto se desplaza el kernel y, por tanto, el tamaño de salida.
- **Padding:** controla el tratamiento de los bordes y las dimensiones de salida.
- **Dilation:** permite ampliar el campo receptivo del kernel sin aumentar su número de pesos.

Canales de entrada, filtros de salida y canales de salida

1 Canales de entrada (in_channels)

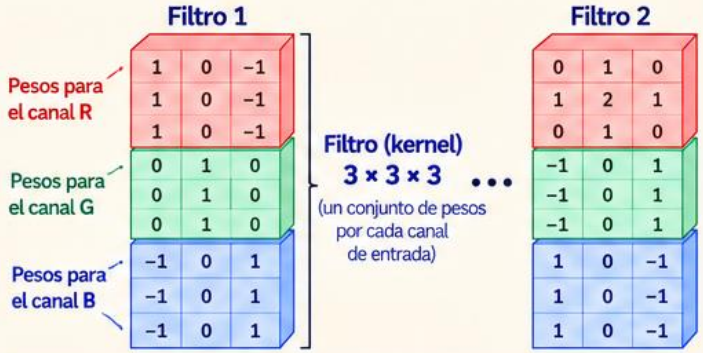
Número de canales que recibe la capa. Por ejemplo, una imagen RGB tiene 3 canales: rojo, verde y azul.



Imagen de entrada ($H \times W \times 3$)
in_channels = 3

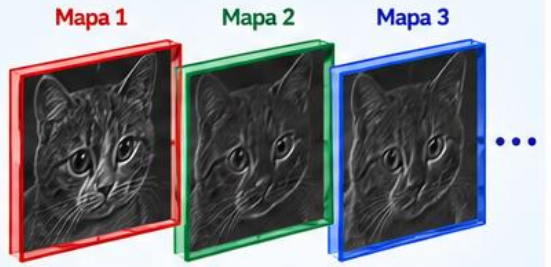
2 Filtros de salida

Cada filtro es un conjunto de pesos aprendibles que se aplica sobre todos los canales de entrada para detectar un tipo de característica (por ejemplo, bordes, texturas o formas).



3 Canales de salida (out_channels)

Número de mapas de características generados por la convolución. Es igual al número de filtros definidos en la capa.



Mapas de características de salida
($H_{out} \times W_{out} \times 3$)
out_channels = 3

Proceso de convolución con múltiples canales

En una convolución ordinaria, cada filtro procesa todos los canales de entrada y genera un mapa de características.



Resumen

- **Canales de entrada (in_channels):** número de canales que recibe la capa (ej. 3 para una imagen RGB).
- **Filtros de salida:** cada filtro es un conjunto de pesos que se aplica sobre todos los canales de entrada para detectar un tipo de característica.
- **Canales de salida (out_channels):** número de mapas de características generados por la convolución (igual al número de filtros).
- **En una convolución ordinaria,** cada filtro procesa todos los canales de entrada y genera un mapa de características.



Impacto del tamaño de la imagen y del batch en una capa convolucional

1 ¿El número de parámetros depende del tamaño de la imagen?

No. Para una Conv2d estándar, el número de parámetros depende de los canales, el kernel y el bias, no de las dimensiones espaciales de entrada.

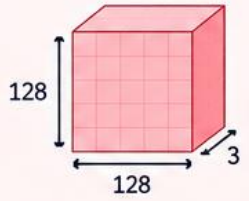
Si duplicamos la resolución de entrada de 128 × 128 a 256 × 256, el número de parámetros de la capa se mantiene igual.



2 ¿Qué sí cambia al aumentar el tamaño de la imagen?

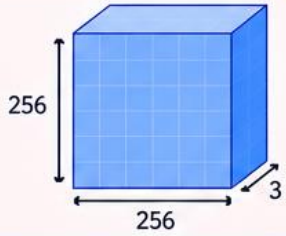
El número de parámetros no cambia, pero sí aumenta el costo computacional y la memoria necesaria para procesar las activaciones.

Entrada 128 × 128 × 3



Menos operaciones y menor memoria.

Entrada 256 × 256 × 3



Más operaciones y mayor memoria (aproximadamente 4× más en este ejemplo).

3 Tamaño del batch

Cada imagen del batch se procesa de manera independiente, pero todas utilizan los mismos filtros y los mismos pesos aprendibles. Los batches no se procesan como canales adicionales.

Representación en PyTorch

Una capa Conv2d recibe un tensor de cuatro dimensiones:

$$(N, C_{in}, H, W)$$

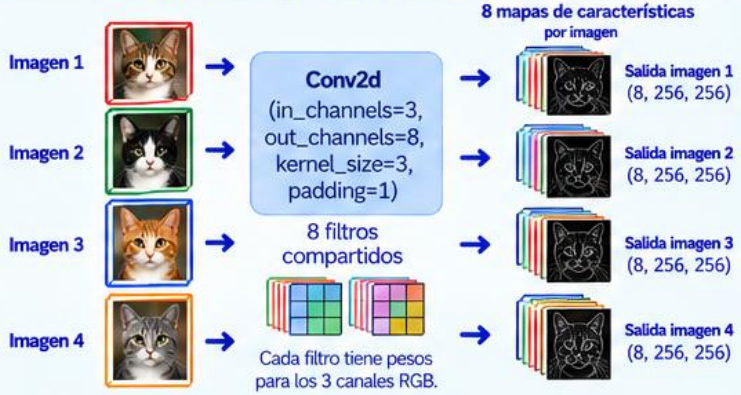
Dimensión	Significado
N	Número de imágenes en el batch
C_{in}	Número de canales por imagen
H	Altura de cada imagen
W	Anchura de cada imagen

Por ejemplo, con un batch de 4 imágenes RGB de 256 × 256 píxeles:

$$(4, 3, 256, 256)$$

4 imágenes independientes, cada una con 3 canales RGB.

Ejemplo de procesamiento de un batch



Tensor de salida del batch:

$$(4, 8, 256, 256)$$

4 ¿El número de parámetros aumenta al incrementar el batch?

No. El número de parámetros depende solo de:

- Canales de entrada (in_channels)
- Canales de salida (out_channels)
- Tamaño del kernel (kernel_size)
- Bias (si se utiliza)

El tamaño del batch no afecta la cantidad de parámetros.

5 ¿Qué sucede durante el entrenamiento?

1. Las cuatro imágenes pasan por la misma red (misma U-Net).
2. Se generan cuatro predicciones de segmentación.
3. Se calcula la pérdida de cada predicción respecto de su máscara.
4. Normalmente, se promedian las pérdidas del batch.
5. Mediante backpropagation, se calculan los gradientes de los parámetros compartidos.
6. El optimizador actualiza los pesos de la red una vez por batch.

Así, los filtros aprenden utilizando información de varias imágenes, sin necesitar un conjunto diferente de pesos para cada una.

Batch Normalization

Normaliza las activaciones para un entrenamiento más estable y eficiente.

1 ¿Qué es?

Batch Normalization (BN) es una capa que normaliza las activaciones de cada canal utilizando las estadísticas del batch (media y varianza).

Su objetivo es reducir el cambio en la distribución de las activaciones durante el entrenamiento, lo que ayuda a que la red converja más rápido y sea más estable.



2 ¿Cómo funciona?

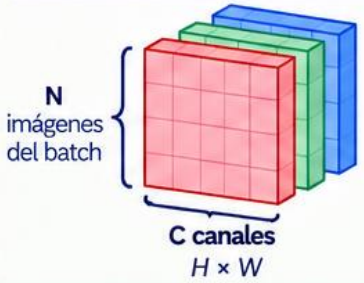
Para cada canal, BN calcula la media y varianza de las activaciones del batch (considerando todas las imágenes y todas las posiciones espaciales), normaliza las activaciones y luego aplica una transformación lineal aprendible (escala y desplazamiento).

3 ¿Por qué se usa?

- Entrenamiento más estable y rápido.
- Permite usar tasas de aprendizaje más altas.
- Reduce el riesgo de saturación de activaciones.
- Actúa como una regularización ligera (efecto de menor overfitting).
- Hace que la red sea menos sensible a la inicialización de pesos.

4 Proceso de normalización (por canal)

Activaciones de entrada



Cálculo de estadísticas

Para cada canal c, se calcula la media (μ) y la varianza (σ^2) utilizando todas las imágenes del batch y todas las posiciones espaciales ($H \times W$).

$$\mu_c \quad \sigma_c^2$$

(por canal)

Normalización

Se resta la media y se divide entre la desviación estándar.

$$\hat{x} = \frac{x - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}}$$

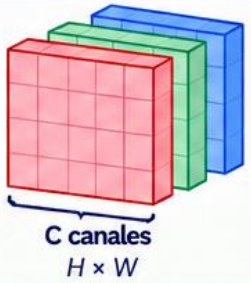
Transformación aprendible

Se aplica una escala (γ) y un desplazamiento (β) aprendibles por canal.

$$y = \gamma_c \hat{x} + \beta_c$$

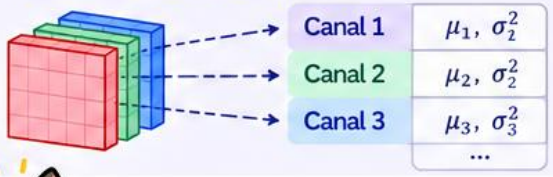
γ y β son parámetros aprendibles.

Activaciones de salida



5 ¿Qué se normaliza?

Las estadísticas (media y varianza) se calculan por canal, usando todas las imágenes del batch y todas las posiciones espaciales.



6 ¿Qué no es?

- No normaliza por imagen individualmente.
- No utiliza filtros con pesos para calcular la media y varianza.
- No cambia el número de canales ni las dimensiones espaciales.

Sí tiene parámetros aprendibles (γ y β) por canal.

7 ¿Cuándo se usa?

Se coloca comúnmente después de la convolución y antes de la función de activación (por ejemplo, ReLU).

Convolución → **Batch Normalization** → **ReLU**



Resumen

- Normaliza las activaciones por canal usando la media y varianza del batch.
- Aplica una transformación lineal aprendible (escala y desplazamiento).
- Facilita un entrenamiento más estable y rápido, y actúa como una regularización ligera.

Group Normalization (GN)

Normaliza las activaciones por grupos de canales, compartiendo las estadísticas dentro de cada grupo.

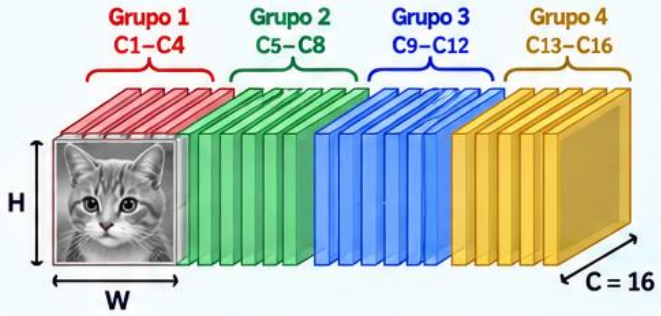
1 ¿Qué es?

Group Normalization (GN) es una capa que normaliza las activaciones dividiendo los canales en grupos. Para cada grupo, calcula la media y varianza utilizando todos los valores de los canales del grupo (en todas las posiciones espaciales), de manera independiente para cada imagen del batch.

A diferencia de Batch Normalization, GN no mezcla estadísticas de distintas imágenes del batch, por lo que funciona bien con batches pequeños.

2 ¿Cómo se dividen los canales?

Ejemplo: 16 canales y 4 grupos
($num_groups = 4$, $num_channels = 16$)



3 Normalización por grupo

Para cada grupo g, se calcula una sola media y una sola varianza utilizando todos los canales del grupo y todas sus posiciones espaciales, por cada imagen del batch.

$$\hat{x}_c = \frac{x_c - \mu_g}{\sqrt{\sigma_g^2 + \varepsilon}}$$

donde

μ_g: media del grupo g
σ_g^2: varianza del grupo g

(calculadas con todos los canales del grupo y todas las posiciones espaciales, por imagen)

4 Parámetros aprendibles por canal

Después de normalizar, cada canal tiene sus propios parámetros de escala (γ_c) y desplazamiento (β_c), aunque comparta las estadísticas con los demás canales de su grupo.

$$y_c = \gamma_c \left(\frac{x_c - \mu_g}{\sqrt{\sigma_g^2 + \varepsilon}} \right) + \beta_c$$

μ_g, σ_g^2: compartidos entre los canales del grupo g
γ_c: parámetro de escala propio del canal c
β_c: parámetro de desplazamiento propio del canal c

5 Ejemplo: 16 canales, 4 grupos

Cada grupo tiene 4 canales. Los canales de un mismo grupo comparten la misma media y varianza, pero cada canal tiene su propio γ y β.

Grupo 1 C1 – C4	Grupo 2 C5 – C8	Grupo 3 C9 – C12	Grupo 4 C13 – C16
μ_1, σ_1^2	μ_2, σ_2^2	μ_3, σ_3^2	μ_4, σ_4^2
C1 (γ_1, β_1)	C5 (γ_5, β_5)	C9 (γ_9, β_9)	C13 $(\gamma_{13}, \beta_{13})$
C2 (γ_2, β_2)	C6 (γ_6, β_6)	C10 $(\gamma_{10}, \beta_{10})$	C14 $(\gamma_{14}, \beta_{14})$
C3 (γ_3, β_3)	C7 (γ_7, β_7)	C11 $(\gamma_{11}, \beta_{11})$	C15 $(\gamma_{15}, \beta_{15})$
C4 (γ_4, β_4)	C8 (γ_8, β_8)	C12 $(\gamma_{12}, \beta_{12})$	C16 $(\gamma_{16}, \beta_{16})$

6 ¿Cuántos valores intervienen?

Ejemplo: 16 canales de 256 × 256, con 4 grupos. Cada imagen del batch tiene:

$$\text{Valores por grupo} = \frac{16}{4} \times 256 \times 256 = \mathbf{262{,}144}$$

Estos 262,144 valores se utilizan para calcular una media y una varianza por grupo (por imagen).

Parámetros aprendibles de la capa:

$$N_{\text{parámetros}} = 2(16) = \mathbf{32}$$

(γ y β por cada canal)

Resumen



- **GN** divide los canales en grupos y normaliza las activaciones usando la media y varianza de cada grupo.
- Las estadísticas se calculan con todos los canales del grupo y todas las posiciones espaciales, por imagen.
- Cada canal tiene sus propios parámetros aprendibles γ y β.
- No mezcla valores de distintas imágenes del batch.
- El número de parámetros depende solo del número de canales (2 por canal) y no del tamaño de la imagen ni del batch.

Funciones de activación: ReLU

1 ¿Qué es una función de activación?

Una función de activación es una transformación matemática aplicada a los valores producidos por una capa neuronal.
Su principal propósito es introducir no linealidad, permitiendo que una red neuronal aprenda relaciones más complejas que las que podrían representarse mediante una sucesión de operaciones lineales.

Sin funciones de activación no lineales, varias capas convolucionales consecutivas, sin otras operaciones no lineales, podrían representarse mediante una única transformación lineal o afín equivalente.

2 ¿Qué es ReLU?

ReLU significa Rectified Linear Unit (unidad lineal rectificada).

$$f(x) = \max(0, x)$$

Es decir:

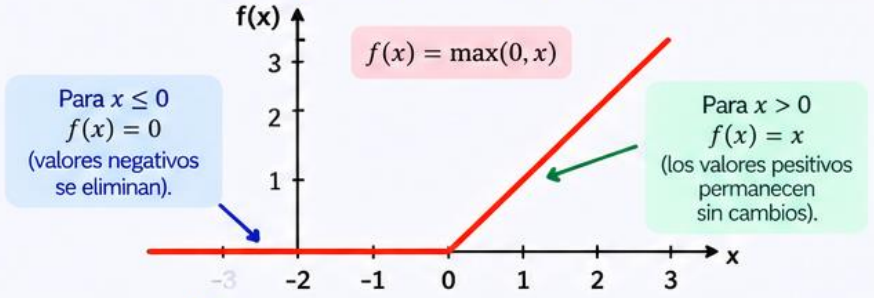
$$f(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

Esto significa que:

- Los valores negativos se convierten en cero.
- Los valores positivos permanecen sin cambios.
- Los valores iguales a cero permanecen en cero.

3 Representación gráfica de ReLU

ReLU elimina los valores negativos y conserva los positivos.



Operaciones de pooling: Max Pooling

1 ¿Qué es pooling?

El pooling es una operación que resume regiones locales de un mapa de características para reducir, habitualmente, sus dimensiones espaciales.

A diferencia de una convolución ordinaria, las operaciones clásicas de pooling no utilizan filtros con pesos aprendibles.
Existen diferentes tipos, como Average Pooling y Max Pooling.
Nos concentraremos en este último.

2 ¿Qué es Max Pooling?

Max Pooling selecciona el valor máximo de cada región local del mapa de características.
Por ejemplo, MaxPool2d(kernel_size=2, stride=2) divide el mapa en regiones de 2 × 2, sin superposición, y conserva únicamente el valor máximo de cada región.

3 Ejemplo de Max Pooling 2 × 2

Entrada: 4 × 4

1	3	2	4
5	6	1	2
7	2	8	1
3	4	0	9

MaxPool2d (kernel_size = 2, stride = 2) →

Salida: 2 × 2

6	4
7	9

Cada región de 2 × 2 se reemplaza por su valor máximo.
La salida conserva cuatro valores en lugar de dieciséis.

Regiones de 2 × 2 y sus máximos

1	3	→ máximo: 6	2	4	→ máximo: 4	7	2	→ máximo: 7	8	1	→ máximo: 9
5	6		1	2		3	4		0	9	

Expresión matemática

$$Y(i,j) = \max_{(m,n) \in R_{i,j}} X(m,n)$$

Donde $R_{i,j}$ es la región local del mapa de características correspondiente a la posición (i, j).

Resumen



- **Función de activación (ReLU):** introduce no linealidad. ReLU convierte valores negativos en cero y mantiene los positivos.
- **Pooling (Max Pooling):** resume regiones locales seleccionando el valor máximo, reduciendo el tamaño espacial del mapa de características.
- **ReLU y Max Pooling:** se utilizan comúnmente en conjunto en las redes convolucionales.

Dropout

Desactiva activaciones aleatoriamente durante el entrenamiento para reducir el sobreajuste.

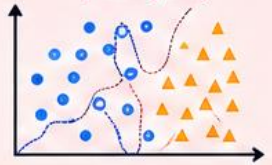


Obliga a la red a aprender representaciones más robustas.

1 ¿Qué es?

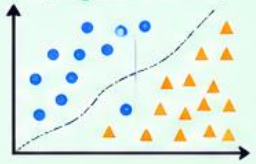
Dropout es una capa de regularización que desactiva aleatoriamente algunas activaciones durante el entrenamiento, reduciendo el riesgo de sobreajuste.

Sin Dropout (sobreajuste)



Aprende demasiado bien los datos de entrenamiento y no generaliza.

Con Dropout (mejor generalización)



Aprende patrones más robustos y generaliza mejor a datos nuevos.

2 ¿Cómo funciona?

Durante el entrenamiento, cada activación tiene una probabilidad p de ser desactivada (puesta en cero). Las activaciones que se conservan se escalan por $1/(1-p)$.

Entrada (4 × 4)

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

$\odot$

Máscara aleatoria (p = 0.25)

1	0	1	1
1	1	0	1
0	1	1	1
1	1	0	1

1 = se conserva
0 = se desactiva

→

Salida (4 × 4)

1.3	0	4.0	5.3
6.7	8.0	0	10.7
0	13.3	14.7	16.0
17.3	18.7	0	21.3

Valores conservados × $1/(1-p)$ (para $p = 0.25 \rightarrow \times 1.333...$)

3 Fórmula

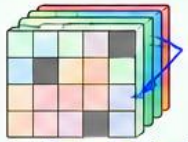
$$y_i = \frac{m_i x_i}{1-p}$$

- x_i: activación original
- m_i: máscara aleatoria (0 o 1)
- p: probabilidad de desactivación

- Si $m_i = 0 \rightarrow y_i = 0$ (se desactiva).
- Si $m_i = 1 \rightarrow y_i = x_i/(1-p)$ (se conserva y se escala).
- Se usa "inverted dropout" para que $E[y_i] = x_i$ durante el entrenamiento.

4 Entrenamiento vs. evaluación

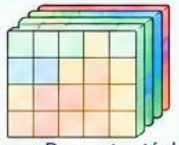
Entrenamiento (model.train())



píxeles desactivados aleatoriamente

- Dropout está activo.
- Desactiva activaciones aleatoriamente.
- Escala las restantes por $1/(1-p)$.

Evaluación (model.eval())



- Dropout está desactivado.
- Todas las activaciones pasan sin cambios.
- No se aplica ningún factor de corrección.

5 Parámetros

Parámetro	Función
p ($p_dropout$)	Probabilidad de desactivar cada activación durante el entrenamiento.
`inplace`	Si es False (recomendado), no modifica directamente el tensor de entrada.

Valores típicos de p

p = 0.1	10 % de desactivación (regularización ligera)
p = 0.3	30 % de desactivación (valor común)
p = 0.5	50 % de desactivación (regularización fuerte, puede causar subajuste si es muy alto)

6 ¿Cuántos parámetros tiene?

Ninguno.

Dropout no tiene pesos, escalas ni desplazamientos que aprender.



Parámetros entrenables = 0

7 Dropout vs. Dropout2d (en redes convolucionales)

Dropout (elementos individuales)



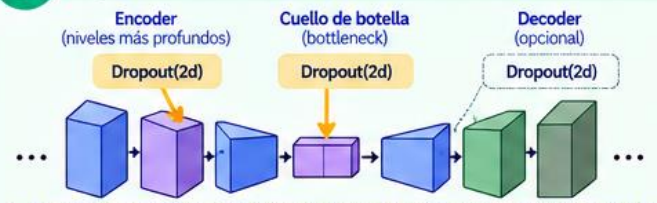
Desactiva activaciones individuales (pixeles) en todo el tensor. Cada elemento tiene probabilidad p de ser puesto en cero.

Dropout2d (canales completos)



Desactiva mapas de características completos (canales). Cada canal tiene probabilidad p de ser puesto en cero.

8 ¿Dónde usar Dropout en U-Net?



- Suele ser más efectivo en los niveles profundos del encoder o en el cuello de botella.
- No es obligatorio usarlo después de todas las convoluciones.
- El valor de p debe evaluarse según las métricas de entrenamiento y validación.

9 Características importantes

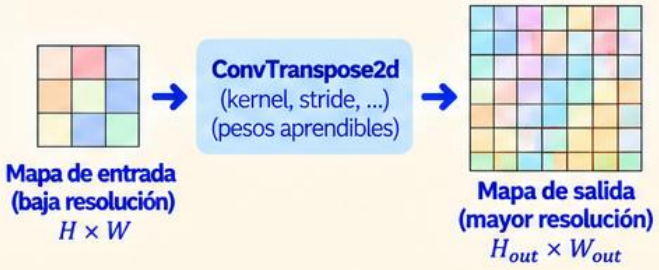
- ✓ Reduce el sobreajuste.
- ✓ No cambia las dimensiones del tensor.
- ✓ No tiene parámetros entrenables.
- ✓ Los valores conservados se escalan por $1/(1-p)$ durante el entrenamiento.
- ✓ Se desactiva automáticamente en evaluación (model.eval()).
- ✗ El valor de p demasiado alto puede provocar subajuste.

ConvTranspose2d

Permite aumentar la resolución espacial de los mapas de características.

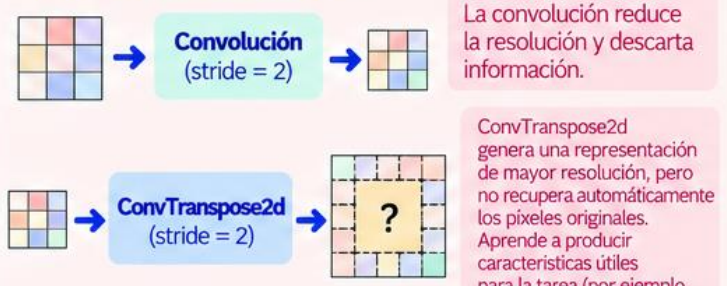
1 ¿Qué es?

ConvTranspose2d es una capa entrenable que construye mapas de caracteristicas de mayor resolución a partir de un mapa de entrada de menor resolución. Se utiliza principalmente en el decoder de arquitecturas como U-Net.



2 No es la operación inversa

Aunque suele llamarse convolución transpuesta o deconvolución, ConvTranspose2d no es la inversa de una convolución convencional.



3 ¿Cómo funciona?

1. Inserta valores entre las posiciones de la entrada (según el stride).
2. Aplica una convolución con pesos aprendibles sobre esta entrada expandida.
3. Produce un mapa de características de mayor resolución.



4 Cálculo de la dimensión de salida

Para una convolución transpuesta bidimensional, la altura de salida se calcula como:

$$H_{out} = (H_{in} - 1)S - 2P + D(K-1) + OP + 1$$

La anchura W_{out} se calcula con la misma expresión, utilizando los valores correspondientes a esa dimensión.

H_{in}	Altura del tensor de entrada.
K	Tamaño del kernel en la dimensión analizada.
S	Stride: factor que determina la separación espacial entre las contribuciones de posiciones consecutivas de entrada.
P	**Padding:** ajusta las dimensiones de salida según la convención de la convolución transpuesta.
D	Dillation: separación entre los elementos del kernel.
OP	**Output padding:** ajuste adicional de la dimensión de salida.
H_{out}	Altura del tensor de salida.

5 Ejemplo numérico

Entrada: 4×4
Kernel: 3×3
Stride: 2, Padding: 1
Dillation: 1, Output padding: 1

$$\begin{aligned} H_{out} &= (4-1)2 - 2(1) + 1(3-1) + 1 + 1 \\ &= 3 \cdot 2 - 2 + 2 + 1 + 1 \\ &= 8 \end{aligned}$$



6 ¿Qué es output padding (OP)?

El output padding es un ajuste adicional de la dimensión de salida que se utiliza para resolver ambigüedades de tamaño.

- No implica necesariamente agregar valores cero al tensor resultante.
- Solo afecta el tamaño calculado de la salida, normalmente añadiendo entre 0 y $(stride - 1)$ posiciones adicionales.
- Se usa cuando se requiere que la salida tenga un tamaño específico (por ejemplo, para que coincida con una conexión de salto en U-Net).

Ejemplo: efecto de output padding
(stride = 2, kernel = 3, padding = 1, dillation = 1)



Resumen



- **ConvTranspose2d** aumenta la resolución espacial de los mapas de características.
- No es la operación inversa de una convolución convencional; es una operación entrenable.
- La dimensión de salida se calcula con $H_{out} = (H_{in} - 1)S - 2P + D(K-1) + OP + 1$.
- El *output padding* ajusta el tamaño de la salida; no agrega ceros necesariamente.